

Stateful VM Migration Among Heterogeneous WebAssembly Runtimes for Efficient Edge-cloud Collaborations

Daigo Fujii
Future University Hakodate
Hokkaido, Japan
b1020144@fun.ac.jp

Katsuya Matsubara
Future University Hakodate
Hokkaido, Japan
matsu@fun.ac.jp

Yuki Nakata
SAKURA internet Inc.
Hokkaido, Japan
y-nakata@sakura.ad.jp

ABSTRACT

WebAssembly (Wasm) has been attracting attention as a common platform for edge computing thanks to its architecture neutrality, sandboxing for security, and lightweight characteristics that fit the requirements for distributed and cooperated processing with heterogeneous compute nodes, which consist of edge devices and clouds. Virtual machine (VM) migration technology enables seamless and flexible workload offloading on edge-cloud collaboration, but unfortunately, VM migration among different Wasm runtimes is complicated since the Wasm standardization does not specify the implementation design of internal runtime very well. Nevertheless, many implementations of Wasm runtime already exist, and each has different characteristics. We therefore aim to implement a stateful VM migration among most of the major Wasm runtimes. In this work, we prototyped a VM migration mechanism between WasmEdge and WAMR to investigate the technical challenges and determine the feasibility of implementing the runtime-neutral VM migration. Our experimental results on heterogeneous VM migration demonstrate the value of coexisting multiple runtimes well-suited for each edge and cloud and seamlessly offloading the workloads among them.

CCS CONCEPTS

• **Computing methodologies** → **Distributed computing methodologies**; • **Software and its engineering** → **Software infrastructure**; *Cloud computing*; *Embedded software*.

KEYWORDS

Application checkpointing, live migration, Wasm

ACM Reference Format:

Daigo Fujii, Katsuya Matsubara, and Yuki Nakata. 2024. Stateful VM Migration Among Heterogeneous WebAssembly Runtimes for Efficient Edge-cloud Collaborations. In *7th International Workshop on Edge Systems, Analytics and Networking (EdgeSys '24)*, April 22, 2024, Athens, Greece. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3642968.3654816>

1 INTRODUCTION

In the edge devices, edge clouds, and data-center cloud that comprise edge computing (also known as fog computing [2]), there is a

trade-off between computing resources and performance. Therefore, it is essential to determine where application workloads should be deployed in accordance with each node's computational capacity and the application's responsiveness requirements. This can be difficult, however, because the situation may be constantly changing: for example, when mobile devices such as cars or mobile phones are moving, the distance to the collaborating edge cloud node grows farther away, and communication latency will increase unless the corresponding node is switched to a neighboring one. Also, the devices may experience poor network conditions or be out of network range as they move, necessitating that all computation be performed on the device, including workloads offloaded to the cloud.

Virtual Machine (VM) migration technology could be helpful for such scenarios because it allows seamless and flexible workload offloading among edge devices and clouds. However, the stateful VM migration among edge devices and clouds presents two significant technical challenges. The first challenge relates to the process migration among heterogeneous platforms. In the realm of cloud, edge servers, and edge devices, various OSs and CPU architectures can be utilized— for example, x86-64 architectures are prevalent in cloud and edge servers for handling complex computations, while Arm architectures, known for their low power consumption, are commonly used in edge devices. Additionally, a real-time OS like NuttX may be utilized in edge devices. Consequently, stateful migration in edge computing necessitates the ability to absorb these differences. The second challenge is how to reduce the file size and migration time to checkpoint. In edge computing, there are communication areas where the network bandwidth may be small, so to achieve seamless live migration in such an environment, the program state to be transferred should be as small as possible. Furthermore, since real-time applications are often handled in edge computing, minimizing application downtime is desirable as well.

To address these challenges, WebAssembly (Wasm) has frequently been proposed as a live migration infrastructure in edge computing. Wasm is a type of virtual machine that can absorb heterogeneous platforms and is lighter weight than VMs and containers. These characteristics make Wasm suitable as a live migration infrastructure for edge computing. It is also characterized by the existence of various runtimes: for example, Wasm Micro Runtime (WAMR) ¹ specializes in lightweight footprints, while WasmEdge ² excels in cloud-native features and serves as a container backend.

In this study, we aim to enable edge devices and clouds to adopt well-suited runtimes and achieve stateful VM migration among heterogeneous Wasm runtimes to encourage seamless and flexible edge-cloud collaboration. We adopted WasmEdge and WAMR as



This work is licensed under a Creative Commons Attribution-NonCommercial 4.0 License.

EdgeSys '24, April 22, 2024, Athens, Greece

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0539-7/24/04

<https://doi.org/10.1145/3642968.3654816>

¹Wasm Micro Runtime: <https://github.com/bytecodealliance/wasm-micro-runtime>

²WasmEdge: <https://github.com/WasmEdge/WasmEdge>

the primary target runtimes and implemented an interoperable VM migration mechanism for both. We also evaluated the migration overhead and the impact on runtime performance resulting from workload offloading from WAMR on an edge device to WasmEdge in the cloud.

The main contributions of this paper are as follows.

- We introduce a new optimization to circumvent the trade-off between application performance and memory consumption on edge devices and clouds by seamless workload offloading among heterogeneous runtimes.
- We clarify the feasibility of the runtime-neutral VM migration and discuss the technical issues related to the implementation by prototyping functionalities of checkpointing, translating, and restoring the VM state on WasmEdge and WAMR.
- We discuss how to improve the runtime neutrality on the checkpointing and restoring method.
- We demonstrate the benefits of adopting each optimal runtime on edge devices and in the cloud by evaluating the effectiveness of workload offload with runtime switching.

2 WEBASSEMBLY RUNTIMES

There are currently more than 20 different Wasm runtimes, each with different features and goals [1]. WasmEdge is a runtime that supports many cloud-native features and is used as a backend for Wasm containers. WAMR is a runtime focused on reducing the runtime footprint and is intended for use in environments with limited computing resources, such as edge devices. In addition, runtimes such as Wasmachine [5], which executes Wasm programs in kernel space, and WaVe [4], which has powerful verification capabilities, are also being researched.

2.1 Comparison of WasmEdge and WAMR

We evaluated the effectiveness of selecting the optimal runtime for different environments by comparing the performance between different runtimes. The runtimes used in this evaluation were WasmEdge and WAMR. For performance evaluation, we utilized *sqlite-bench*³, which is a benchmarking tool for SQLite. The reason for choosing SQLite is that the deployment of databases in the cloud and on edge devices is gaining attention in the context of edge computing. Since edge computing focuses on placing database systems in the cloud or on devices to protect privacy and improve latency, we utilized *sqlite-bench*, a SQLite benchmark, as the application under evaluation.

Sqlite-bench measures the execution time per query for various operations to SQLite, such as read and write. We set the number of queries to 10,000 to enable measurements under a significant load. Runtime performance and memory usage were utilized as evaluation metrics. Our purpose here was to assess the performance of each runtime, so the machines used for the measurements were only those for edge server environments; however, we believe that the relationship between performance per runtime does not change depending on the execution environment. The runtime performance

was determined by the contents of *sqlite-bench*, while memory usage was measured by the Resident Set Size (RSS).

The experimental results are presented in Fig. 1, where it is clear there was a performance tradeoff between the different runtimes. Runtime performance was higher for WasmEdge, averaging about 4.41 times less execution time per query for each item. In contrast, memory usage was about 2.5 times lower for WAMR. These findings clarify that WasmEdge is intended for use on machines with abundant computing resources (e.g., the cloud), and therefore consumes a lot of memory but achieves fast processing, while WAMR is intended for use on machines with few computing resources (e.g., edge devices), and thus specializes in lightweight runtimes.

3 FEASIBILITY OF RUNTIME-NEUTRAL VM MIGRATION

Our approach to demonstrating the feasibility of runtime-neutral VM migration comprises two steps. First, we designed and prototyped a mechanism of VM migration for two well-known runtimes. This mechanism needs to be able to capture the state of a VM on both runtimes, translate it into a runtime-independent representation, and resume it on another runtime. The prototyping helped us clarify the technical issues with extracting the VM state held by each runtime and the requirements for translating between different internal runtime representations of it. Second, we investigated how to improve the runtime neutrality on the VM state checkpointing and translation.

3.1 Prototyping with WasmEdge and WAMR

As illustrated in Fig. 1, our prototyping adopts both WasmEdge and WAMR because WasmEdge has been tuned for clouds and WAMR has characteristics beneficial for edge devices. Note that while this prototyping only focuses on the standard interpreter-based execution, WasmEdge and WAMR support various ways to execute instructions, including the standard interpreter, an interpreter with optimization by non-standard proprietary instructions, Just-In-Time (JIT) compilation, Ahead-Of-Time (AOT) compilation.

3.1.1 VM Checkpointing and Translation. The program state of Wasm consists of the components that change during execution in the Wasm runtime structure. These are the parts we write to the state checkpoint file during migration. The Wasm runtime structure consists of module instance, program counter, and stacks in accordance with the WebAssembly core specifications [3]. A module instance collects definitions for types, functions, tables, memories, and globals. Wasm runtimes change the contents of tables, memories and globals during execution. Note that in this study we do not consider the migration of tables because they are data structures that store the contents of host environments outside Wasm and are not subject to Wasm migration. The stack consists of a value stack, control stack, and frame stack. To summarize, the program state of Wasm comprises memories and globals in the module instance and program counter, and value, control, and frame stacks in the stack.

The implementation of these program states differs among the heterogeneous Wasm runtimes. To enable migration among these runtimes, we checkpointed each program state in a format that is independent of the runtime implementation.

³*sqlite-bench*: <https://github.com/ukontainer/sqlite-bench>

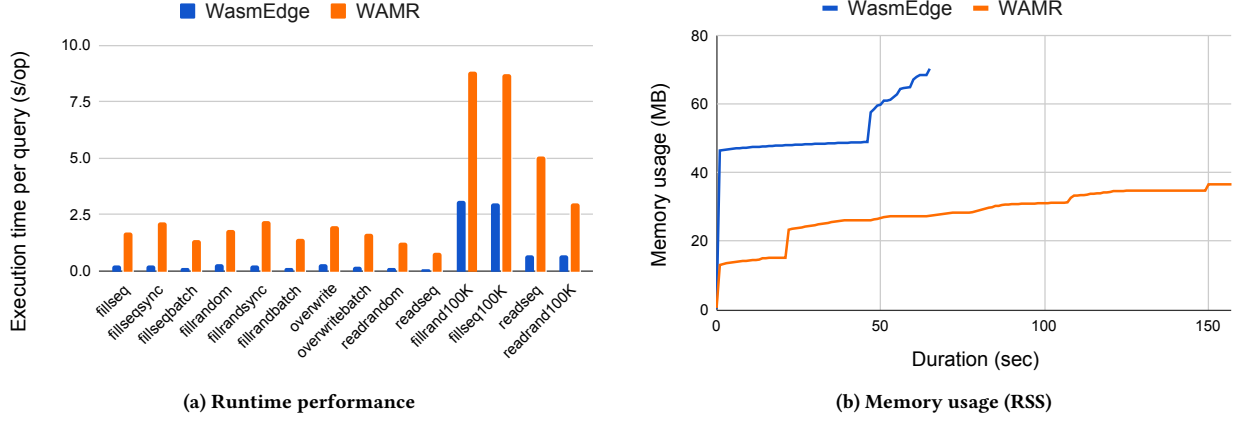


Figure 1: Comparison of WasmEdge and WAMR characteristics

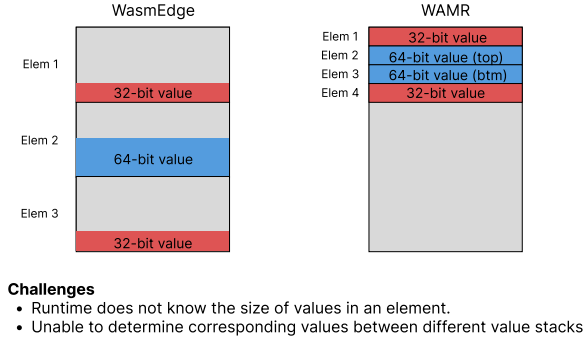


Figure 2: Technical challenges with heterogeneous runtime migration in value stack

Memories and Globals. In WasmEdge and WAMR, there is no discernible disparity in implementation between memories and globals. Therefore, we checkpoint them without translating between the runtimes. According to the WebAssembly core specifications, memories are linear and increase page by page. Hence, the memory state can be restored by extracting only the used contiguous pages rather than the whole region. Globals are an ordered list capable of storing values of 32-bit, 64-bit, or 128-bit types. Given that the type of value is explicitly specified for each index, it is feasible to restore the list in its entirety.

We checkpointed only those areas of memories that had actually been written. To achieve this, we took advantage of the OS kernel’s dirty memory detection feature. Dirty memories pertain to memory regions that have been written to at least once. By reducing the size of the checkpoint, we minimize the overhead associated with transmission and restoration.

Program Counter. There are two main challenges to address in the migration of the program counter. The first is to enable the restoration of absolute addresses of processes in different OSs and different processes. In Wasm, the program counter holds the address of the next Wasm instruction to be executed. Since this value is an

absolute address within the process, it cannot be restored directly during migration.

We addressed this challenge by adding a feature to translate the instruction address into a pair of the function index and the offset from the beginning address of that function. In Wasm, each function in the Wasm code is assigned a unique value, known as the function index. Our idea is to calculate the difference from the beginning address of that function to see if it can be restored.

The second challenge is the need to accommodate the differences in instruction implementation pointed to by the program counter between heterogeneous runtimes. In WasmEdge, instructions are objectified. An instruction object contains various values such as opcode, its arguments, and for jump instructions, the destination address. Thus, the instruction sequence in WasmEdge consists solely of instruction objects. Conversely, in WAMR, instructions are not objectified, and the instruction sequence consists of a mix of instructions and their arguments. This is because WAMR positions the code section of Wasm code directly, without objectifying instructions. Due to these implementation differences, offsets pointing to the same instruction assume distinct values depending on the runtime.

We addressed this challenge by adding functionality to translate each offset mutually. This translation functionality can be computed in advance from the Wasm code, enabling implementation independent of the runtime. Specifically, by sequentially analyzing the Wasm code for each function we can obtain the offset with and without arguments.

Value Stack. The value stack is a stack for storing Wasm values. The Wasm specification specifies that value types can be 32, 64, or 128-bits in size. In WasmEdge, the value stack is implemented with each element being 128 bits so that all types of values can be stored within a single element. If the value to be stored is less than 128 bits, zero padding occurs. The WAMR value stack, on the other hand, is implemented with 32 bits for each element; if a value greater than 32 bits is to be stored, it is stored in 32-bit increments. To restore this different layout of the value stack between runtimes, the internal type information of the value stack is needed. The value stack can be dissolved by value rather than by element from

the value types in the value stack. This allows the value stack to be converted into a common representation. However, the Wasm runtime does not have access to the internal type information of the value stack.

To resolve this challenge, we implemented the type stack, which is a stack that manages the types of values in the value stack. Each time the runtime operates on the value stack, it also operates on the corresponding value type on the type stack, thereby managing the types on the value stack. Note that, this approach incurs an overhead in execution performance because the value and type stacks need to be processed frequently.

Control Stack. The control stack is a stack that manages labels. In Wasm, jump-type instructions such as jumps, branches, and loops that move to a specific instruction address can only move to the location where the label is located. Labels can be generated by specific instructions such as the block instruction. As the name suggests, the label is a block structure, and when a jump instruction is executed within a block, it can move to the end of the block (the location of the end instruction).

WasmEdge does not implement a control stack. Instead, at runtime initialization, pre-processing calculates the instruction addresses to which all jump instructions can move and stores them in the instruction object. This allows jumps without a control stack. WAMR, on the other hand, implements the control stack as specified.

In order to migrate between WasmEdge and WAMR, we need to resolve the lack of a control stack in WasmEdge. However, we do not want to implement the control stack in WasmEdge because it would increase the execution overhead and dependency on runtime.

We solved this problem by utilizing the program counter to restore the control stack. Specifically, we analyze the Wasm code in the function sequentially from the beginning to the current program counter at the checkpoint. We check whether the instruction is a block instruction and then resolve it by performing push/pop operations to the control stack based on that instruction. This approach allows for control stack migration without runtime overhead.

Frame Stack. The frame stack is a stack used to hold the context of the function for each function call. The context includes the value stack, the control stack, and the return address (program counter).

Thus, frame stack migration can be achieved by combining the value stack, control stack, and return address.

3.1.2 Lazy Type Identification for the Value Stack. As stated above, the introduction of a type stack imposes a significant overhead on the execution performance. We therefore propose an optimization technique to identify the type of value stack at any given runtime point without any runtime overhead.

Our proposed technique generates a type table for the value stack using only the Wasm code. This technique enables the user to easily identify the type of the value stack at any instruction execution point, without any additional performance overhead. This is because Wasm ensures that the type of the value stack is uniquely determined at every instruction execution point. Each instruction and label of the block structures in Wasm is associated with a predetermined type for value stack operations. For example, the *i32.add* instruction takes [i32, i32] as arguments and returns

[i32]. Control instructions such as jump, if, and loop can only move to labels at the end of blocks.

From these two characteristics, the type of the value stack can be uniquely determined at every execution point in Wasm. Without considering control instructions, all instructions are executed sequentially, and thus the type of the value stack is uniquely determined at any execution point. Next, we consider the control instructions: jump, if, and loop. In jump instructions, they can only jump to endpoints of nested blocks (in other words, labels) in Wasm. Therefore, even if multiple instructions are skipped by a jump, the type in the middle is not indeterminate.

The if instructions can be classified into two types: if-else-end and if-end. The if-else-end type is described as *if bt instr₁* else instr₂* end*. Depending on the condition, either *instr₁** or *instr₂** is executed, but both follow the block type defined by *bt*. Therefore, after the branch, the type of the value stack is uniquely determined. The if-end type corresponds to the if-else-end type when *instr₂** is null. Consequently, *bt* is also null, satisfying the type $[\] \rightarrow [\]$, and the type of the value stack is uniquely determined even after the branch. For these reasons, the type of the value stack is determined even in the presence of branching instructions.

The loop instruction can be expressed as *val^m loop bt instr* end* and expanded to *label_m loop bt instr* end val^m instr* end*. *label_m instr** implies that when the block is terminated by a jump instruction, the value *m* is returned and *instr** is executed. In other words, when returning to the beginning of the loop, the value stack always has the type specified by the argument. Therefore, the type of the value stack within the block remains unchanged regardless of the number of iterations of the loop. Consequently, the type of the value stack is clearly determined at each execution point.

3.2 Discussion on Runtime Neutrality

Above, we presented our prototype of the heterogeneous runtime migration between WasmEdge and WAMR. On the basis of the knowledge gained from this prototype, we here discuss the design of a runtime-neutral heterogeneous inter-Wasm runtime migration.

We found that the memory and globals of module instances have no implementation differences at runtime. Therefore, we believe that saving other execution states to memory or global can be achieved. Saving the execution state to memory or global can be done in a runtime-neutral manner by using the Wasm instructions to save it, since all Wasm runtimes can execute Wasm code. Migration can also be done runtime-neutrally by implementing a function outside the runtime to absorb implementation differences.

Wasm has two instructions, store and load, to store/retrieve elements of the value stack to/from memory. By using these instructions, the value stack can be stored in memory. In addition, since the type of the value stack can be specified by using a type table, differences in the type of the value stack can be absorbed. In this way, we believe the value stack can be transitioned to runtime-neutral. The control stack can be traced back to its point of execution using the Wasm code if the program counter and return address (i.e., instruction address) are known. Therefore, if the instruction address can be transitioned to runtime-neutral, the control stack can also be transitioned to runtime-neutral. The frame stack consists of the value stack, control stack, and return address. If the instruction address

Table 1: Hardware and middleware specifications

ISA	x86-64	Aarch64
OS	Ubuntu 22.04.3	Debian GNU/Linux 11
Kernel version	6.5.0	6.1.77
CPU model	Intel i7-10700K	Broadcom BCM2711
RAM	32 GB	4GB

Table 2: Checkpointed program state size (KB)

Program	WebAssembly	CRIU
binary-trees	2124.889	4326.082
n-body	22.509	131.704
sqlite	202.405	226.193

can be transitioned to runtime-neutral, the value stack, control stack, and return address can all be transitioned to runtime-neutral, and thus the frame stack can also be transitioned to runtime-neutral. On the basis of these considerations, we believe that if instruction addresses can be migrated to runtime-neutral, Wasm migration can be achieved using a runtime-neutral approach.

4 EVALUATION

We evaluated the feasibility of efficient edge-cloud collaboration by measuring the overhead of the migration mechanism on execution performance and the impact of edge-cloud collaboration through migration on application performance using the implemented prototypes. These measurements were performed using the edge devices and cloud servers listed in Table 1. In this paper, we refer to the cloud server as x86-64, and the edge device as Aarch64. The Wasm-compilation of the program used the wasi-SDK tool chain.

4.1 Migration Overhead

4.1.1 Overhead of Checkpointing and Restoring. We utilized the time taken to the checkpoint, the time taken to restore, and the state size of the checkpointed program to measure the migration overhead. A comparison to our prototype was made with the Linux process migration software Checkpoint/Restore In Userspace (CRIU). The applications used for evaluation were the binary-trees and n-body benchmark programs from The Computer Language Benchmarks Game and SQLite, a practical application program.

The experimental results are shown in Fig. 3 and Table 2. As we can see, our prototype succeeded in better reducing the checkpointing time and the size of the dumped state file compared to CRIU, but the restoring time for SQLite was worse. In terms of the checkpoint time, our method was faster than CRIU, improving the performance by 30 to 100 times. The size of the program state was 2.1 MB for our method and 4.3 MB for CRIU for the binary tree, and 22 KB for our method and 131 KB for CRIU for the n-body. The checkpoints for SQLite were 202KB for our method and 226KB for CRIU. In the case of checkpointing binary-trees and n-body, the size was sufficiently smaller than that of CRIU, and in the case of checkpointing SQLite, the size was about the same. Therefore, the size of the files checkpointed by this method is sufficiently small.

Table 3: Execution performance overhead (%)

Platform	Execution time	Memory usage
WasmEdge on x86-64	114	101
WasmEdge on Aarch64	117	100
WAMR on x86-64	105	97
WAMR on Aarch64	102	100

Table 4: Execution time for each CPU/runtime (sec)

Runtime	x86-64	Aarch64
WasmEdge	79.21	347.234
WasmEdge with migration	92.64	409.085
WAMR	170.45	661.491
WAMR with migration	173.85	668.189

The reason the SQLite restore time is slower than other applications is that the application startup time accounts for most of the restore time. In fact, with the WasmEdge on the edge device, 348.2 milliseconds of the 349.2 milliseconds restore time was boot time, and with the WAMR on the cloud server, 3.82 milliseconds of the 3.89 milliseconds restore time was boot time. Startup time here refers to the time from when the application code is loaded and verified by the runtime until just before the first Wasm instruction is executed. Since the startup time is directly related to the restore time, a large application such as SQLite, which takes a long time to start up, will also take a long time to restore. The extremely long time required is only for edge devices, and restoration on cloud servers takes the same amount of time or several times as long as CRIU restoration time. These results indicate that we need to come up with ways to further reduce the time required for restoration.

4.1.2 Slowdown in Application Performance. We achieved live migration by adding an implementation to the runtimes, WasmEdge and WAMR. We compared the execution performance of the original and modified runtimes to see if this modification worsened execution performance. Evaluations were performed in the same way as in Experiment 5.1. Evaluation items were the total execution time and memory usage (RSS).

Table 3 lists the results, where we can see that there was little impact on runtime performance and no change in memory usage for either the WasmEdge or WAMR runtimes. Execution performance and execution speed showed a 2–5% increase for WasmEdge, a sufficiently small overhead. On the other hand, WasmEdge showed an increase of 14–17%, which is a significant overhead.

However, since WasmEdge is used on a cloud server with a high computing performance, we believe the overhead impact is negligible. In fact, as shown in Table 4, WasmEdge running on x86-64 machines takes 13 seconds, which is not much different from the 7 second difference in increased execution time when running WAMR on Aarch64 machines. Therefore, we conclude that the migration of this method has only a small overhead.

4.2 Impact on Edge-cloud Collaboration

We measured the performance in the case of switching the optimal runtime by migration using the implemented prototype and in the

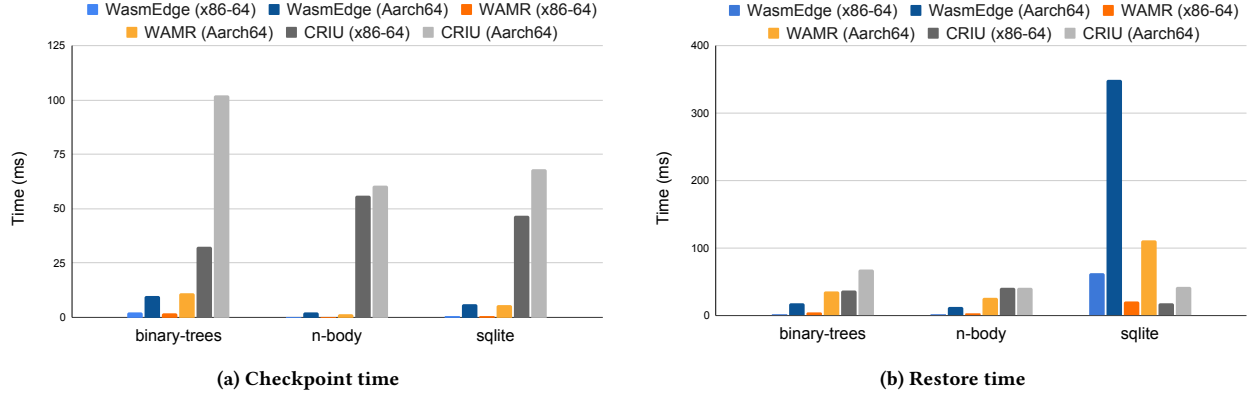


Figure 3: Migration performance

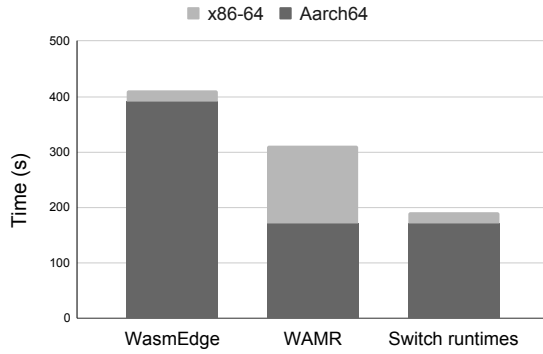


Figure 4: Advantage of offloading device workload to another runtime in the cloud

case of migrating between single runtimes. The application utilized for the evaluation was *sqlite-bench*, which ran 10000 queries. The migration scenario was to initialize *sqlite-bench* on the edge device and migrate the heavily loaded benchmark process to the cloud server for execution. This scenario was compared with migration between the same runtime and migration between different runtimes. In the case of switching runtimes, WAMR was used on the edge device and WasmEdge on the cloud server.

The results are shown in Fig. 4 where we can see that the total runtime of the application performed best when the optimal runtime was selected and migrated. Note that the time it takes to checkpoint the VM state at an Arch64 device, transfer it to the x86-64 cloud, and restore it remains almost constant regardless of the underlying Wasm runtimes, so these times do not consist of the measurement. We believe that the slow initialization process of WasmEdge's *sqlite-bench* is due to the internal memory instance implementation. *sqlite-bench*'s initialization process internally allocates memory space many times in order to generate random data for use in the benchmark, and the process is executed many times to allocate the memory area internally to generate random data for use in the benchmark. We compared the average execution time between WasmEdge and WAMR for *memory.grow*, an instruction

that extends the memory instance area, and found that WasmEdge was slower, by about 13 ms. This is very slow, representing a difference of about 100–10,000 times compared to the other instructions. Therefore, switching the optimal runtime depending on the process showed better results than continuing to operate with a single runtime.

5 CONCLUSION

In cloud-edge collaboration, offloading or balancing workloads among edge devices and clouds by means of the VM migration can potentially improve system response times, availability, and reliability. Wasm can bridge the architectural differences between cloud and edge, but the different runtimes suitable for edge devices and clouds make stateful VM migration challenging. Our prototyping of the VM migration between the WasmEdge and WAMR runtimes in this work has provided insight into minimizing runtime dependencies and shows feasibility.

As our study is still preliminary, the following challenges remain to be addressed. First, the VM state might depend on the state kept outside of the runtime, which needs to be explored. For example, if a Wasm application utilizes the network sockets accessed through the WASI interface, it would need to keep the connection state in the OS kernel rather than the runtime. Second, we want to achieve the migration of stateful VMs that are optimized with runtime-proprietary code optimization and converted with JIT/AOT compilation.

REFERENCES

- [1] Stephen Akinyemi. 8 Oct 2018. Awesome WebAssembly Runtimes. <https://github.com/appcypher/awesome-wasm-runtimes>.
- [2] Flavio Bonomi, Rodolfo Milito, Jiang Zhu, and Sateesh Addepalli. 2012. Fog computing and its role in the internet of things. In *Proc. of the First Edition of the MCC Workshop on Mobile Cloud Computing (MCC '12)*. Association for Computing Machinery, 13–16. <https://doi.org/10.1145/2342509.2342513>
- [3] WebAssembly Community Group. 25 October 2023. WebAssembly Core Specification. https://webassembly.github.io/spec/core/_download/WebAssembly.pdf.
- [4] Evan Johnson, Evan Laufer, Zijie Zhao, Dan Gohman, Shravan Narayan, Stefan Savage, Deian Stefan, and Fraser Brown. 2023. WaVe: a verifiably secure WebAssembly sandboxing runtime. In *2023 IEEE Symposium on Security and Privacy (SP)*. 2940–2955. <https://doi.org/10.1109/SP46215.2023.10179357>
- [5] Elliott Wen and Gerald Weber. 2020. Wasmachine: Bring IoT up to Speed with A WebAssembly OS. In *Proc. of 2020 IEEE Int. Conf. on Pervasive Computing and Communications Workshops (PerCom Workshops)*. 1–4. <https://doi.org/10.1109/PerComWorkshops48775.2020.9156135>